

Documentation Technique - Gaïa's Guardian

Version: Finale (18 avril 2025)

Équipe: Autimatique

Projet: Gaïa's Guardian

Table des matières

Table des matières

1. Introduction1

2. Architecture Logicielle2

3. Systèmes de Jeu3

4. Design Patterns et Bonnes Pratiques12

5. Gestion des Assets28

6. Optimisation et Performance34

7. Système de Compilation et Déploiement.....47

8. Tests et Debugging.....54

9. API de Référence63

10. Annexes70

1. Introduction

1.1 À propos de Gaïa's Guardian

Gaïa's Guardian est un jeu de plateforme-action en 2D où le joueur incarne Gaïa, une entité mystique née des vestiges du monde naturel, qui combat des machines corrompues pour restaurer l'équilibre écologique. Le jeu met l'accent sur la fluidité des mouvements, la richesse du combat et l'évolution dynamique de l'environnement.

1.2 Objectifs de la documentation

Cette documentation technique vise à fournir aux développeurs actuels et futurs une compréhension complète de l'architecture, des systèmes et des outils utilisés dans le développement de Gaïa's Guardian. Elle servira de référence pour la maintenance, l'extension et l'évolution du projet.

1.3 Technologies et outils utilisés

- Moteur de jeu: Unity 2023.2
- Langage de programmation: C#
- Versionnement: Git, GitHub
- Gestion de projet: Asana

- **Communication:** Slack
 - **Audio:** FMOD
 - **Outils graphiques:** Aseprite, Photoshop, Blender
-

2. Architecture Logicielle

2.1 Vue d'ensemble

L'architecture de Gaïa's Guardian suit le modèle MVC (Modèle-Vue-Contrôleur) adapté au développement de jeux vidéo, avec une forte séparation des responsabilités pour garantir maintenabilité et évolutivité.

GaiaGuardian/

```
├── Assets/
|
| ├── Scripts/
| |
| | ├── Core/      # Cœur du système de jeu
| | ├── Controllers/ # Contrôleurs de gameplay
| | ├── Models/    # Modèles de données
| | ├── Views/     # Composants d'affichage
| | ├── Utils/     # Utilitaires génériques
| | ├── AI/        # Intelligence artificielle
| | ├── Audio/     # Gestion audio
| | └── UI/        # Interface utilisateur
|
| ├── Prefabs/     # Préfabriqués
|
| ├── Scenes/      # Scènes du jeu
|
| ├── Graphics/    # Assets graphiques
| |
| | ├── Characters/ # Sprites des personnages
| | ├── Environment/ # Éléments d'environnement
| | ├── Animations/ # Animations
| | └── FX/        # Effets visuels
|
| ├── Audio/       # Assets audio
| |
| | ├── Music/     # Musiques
| | └── SFX/       # Effets sonores
```

```
| | └─ FMOD/      # Projets FMOD
| └─ Resources/   # Ressources chargées dynamiquement
```

2.2 Diagramme de classes principal

Note: Le diagramme complet UML est disponible dans l'annexe A.

2.3 Flux de données

Le flux de données dans Gaïa's Guardian suit un modèle de communication basé sur les événements pour minimiser les dépendances directes entre les composants:

1. Les **entrées utilisateur** sont captées par le InputManager
2. Le PlayerController interprète ces entrées et met à jour le PlayerModel
3. Les changements d'état déclenchent des événements
4. Les systèmes abonnés (combat, animation, audio, etc.) réagissent à ces événements
5. Le GameManager coordonne les systèmes de haut niveau (progression, niveaux, etc.)

2.4 Initialisation du jeu

Séquence d'initialisation:

1. Chargement des services essentiels via ServiceLocator
2. Initialisation des managers système (audio, sauvegarde, etc.)
3. Chargement des données de progression (si une sauvegarde existe)
4. Initialisation du niveau actuel et des entités
5. Démarrage de la boucle de jeu

3. Systèmes de Jeu

3.1 Système de personnage joueur

3.1.1 Structure du personnage

```
public class PlayerController : MonoBehaviour
{
    [SerializeField] private PlayerModel _playerModel;
    [SerializeField] private PlayerView _playerView;

    private StateMachine _stateMachine;
    private MovementSystem _movementSystem;
```

```
private CombatSystem _combatSystem;

private AbilitySystem _abilitySystem;


// Implémentation...
}
```

Le personnage de Gaïa utilise une structure modulaire:

- PlayerModel: Contient toutes les données d'état et attributs
- PlayerView: Gère la représentation visuelle et les animations
- PlayerController: Coordonne la logique et les systèmes

3.1.2 Système de mouvements

Le système de mouvement de Gaïa est basé sur un contrôleur de physique personnalisé pour une meilleure précision que le système standard Unity.

Caractéristiques principales:

- Détection de collision en forme de capsule
- Traitement spécial des plateformes et surfaces
- Système avancé de tampons d'entrée pour les sauts
- Gestion des états de mouvement (air, sol, accroche murale, etc.)

```
public class MovementSystem
{
    private PlayerModel _model;
    private Rigidbody2D _rigidbody;
    private PhysicsController _physicsController;


    // Paramètres de mouvements configurables
    public float MoveSpeed = 8f;
    public float JumpForce = 18f;
    public float DashDistance = 5f;
    public float WallSlideSpeed = 2f;


    // Implémentation...
}
```

3.1.3 Système de combat

Le système de combat gère les attaques de Gaïa et les interactions avec les ennemis:

- Gestion des hitboxes pour différentes attaques
- Calcul des dégâts et effets
- Chaînes de combos et timing
- Effets visuels et feedback

```
public class CombatSystem
{
    private PlayerModel _model;
    private AttackDatabase _attackDatabase;

    // Gestion des attaques et combos
    private Dictionary<AttackType, Attack> _attacks;
    private ComboHandler _comboHandler;

    // Implémentation...
}
```

3.1.4 Système de capacités

Le système de capacités gère les pouvoirs spéciaux de Gaïa:

- Acquisition progressive de nouvelles capacités
- Gestion des coûts en énergie
- Cooldowns et restrictions d'utilisation
- Effets sur l'environnement

```
public class AbilitySystem
{
    private PlayerModel _model;
    private Dictionary<AbilityType, Ability> _abilities;

    // Gestion de l'énergie et des cooldowns
    public float CurrentEnergy { get; private set; }
    private Dictionary<AbilityType, float> _cooldowns;
```

```
// Implémentation...  
}
```

3.2 Système d'IA et ennemis

3.2.1 Architecture de l'IA

L'IA des ennemis est basée sur une machine à états finie (FSM) avec des comportements modulaires:

```
public class EnemyController : MonoBehaviour  
{  
    [SerializeField] private EnemyModel _enemyModel;  
    [SerializeField] private EnemyView _enemyView;  
  
    private StateMachine _stateMachine;  
    private Dictionary<Type, IEnemyBehavior> _behaviors;  
  
    // Implémentation...  
}
```

Types d'états communs:

- IdleState: L'ennemi est inactif
- PatrolState: L'ennemi patrouille dans une zone définie
- AlertState: L'ennemi a détecté le joueur mais n'est pas encore en combat
- ChaseState: L'ennemi poursuit activement le joueur
- AttackState: L'ennemi exécute une attaque
- StunnedState: L'ennemi est temporairement neutralisé
- DeadState: L'ennemi est vaincu

3.2.2 Détection et perception

Le système de perception permet aux ennemis de détecter Gaïa:

- Détection visuelle par raycasting
- Détection de proximité par zones de déclenchement
- Système d'alerte avec mémoire de la dernière position connue
- Communication entre ennemis (alerte de groupe)

```
public class PerceptionSystem  
{
```

```

private EnemyModel _model;

private Transform _target;


public bool CanSeeTarget { get; private set; }
public Vector2 LastKnownPosition { get; private set; }


// Méthodes de détection
public bool CheckLineOfSight(Transform target, float viewDistance, float viewAngle)
{
    // Implémentation...
}


// Implémentation...
}

```

3.2.3 Pathfinding

Le système de navigation utilise une combinaison de:

- Grille de navigation pour le pathfinding global
- Raycast pour la navigation locale et l'évitement d'obstacles
- Points de cheminement pour les patrouilles prédéfinies

```

public class NavigationSystem
{
    private EnemyModel _model;
    private GridPathfinder _pathfinder;


    private List<Vector2> _currentPath;
    private int _currentWaypointIndex;


    // Implémentation...


    public List<Vector2> FindPath(Vector2 start, Vector2 end)
    {

```

```
        // Implémentation du pathfinding...
    }
}
```

3.2.4 Types d'ennemis spécifiques

Chaque type d'ennemi hérite d'une classe de base mais implémente des comportements spécifiques:

```
// Base abstraite pour tous les ennemis
public abstract class Enemy : MonoBehaviour
{
    // Propriétés communes

    // Méthodes virtuelles à implémenter
    protected virtual void InitializeStateMachine() {}
    protected virtual void RegisterBehaviors() {}
    protected virtual void OnDamage(DamageInfo damageInfo) {}
}

// Implémentations spécifiques
public class DroneEnemy : Enemy
{
    // Implémentations spécifiques...
}

public class SentinelEnemy : Enemy
{
    // Implémentations spécifiques...
}

public class BossEnemy : Enemy
{
    // Systèmes supplémentaires pour les phases de boss
    private BossPhaseManager _phaseManager;
```



```
// Implémentations spécifiques...  
}
```

3.3 *Système de niveaux et d'environnement*

3.3.1 Structure des niveaux

Les niveaux de Gaïa's Guardian sont structurés en utilisant:

- Tilemaps Unity pour les éléments de base
- Objets interactifs instanciés comme préfabriqués
- Déclencheurs pour les zones événementielles
- Système de portails pour la connexion entre niveaux

```
public class LevelManager : MonoBehaviour  
{  
    // Références aux composants du niveau  
    [SerializeField] private Transform _playerSpawnPoint;  
    [SerializeField] private List<GameObject> _enemySpawnPoints;  
    [SerializeField] private List<EnvironmentController> _environmentControllers;  
  
    // Gestion de l'état du niveau  
    private Dictionary<string, bool> _checkpoints;  
    private float _purificationLevel;  
  
    // Implémentation...  
}
```

3.3.2 *Système de purification*

Le système de purification transforme dynamiquement l'environnement:

```
public class PurificationSystem : MonoBehaviour  
{  
    // Zones de purification  
    [SerializeField] private List<PurificationZone> _purificationZones;  
  
    // État global de purification  
    private float _globalPurificationLevel;
```

```

private Dictionary<string, float> _zonePurificationLevels;

// Événements

public event Action<float> OnGlobalPurificationChanged;

public event Action<string, float> OnZonePurificationChanged;

// Implémentation...
}

```

La purification affecte:

- L'apparence visuelle (via des shaders de transition)
- Les objets du décor (transformation des assets)
- Les comportements des ennemis (affaiblissement)
- L'accessibilité de certaines zones

3.3.3 Interaction avec les éléments du décor

Les objets interactifs suivent une interface commune:

```

public interface IInteractable
{
    bool CanInteract(PlayerController player);
    void OnInteract(PlayerController player);
    void DisplayInteractionPrompt(bool display);
}

```

Types d'objets interactifs:

- PurificationNode: Points de purification
- Collectible: Objets à collecter
- PlatformSwitch: Interrupteurs activant des plateformes
- EnvironmentHazard: Pièges et dangers
- ClimbableWall: Surfaces d'escalade

3.4 Système de progression

3.4.1 Déblocage des capacités

```

public class ProgressionManager : MonoBehaviour
{
    // Capacités débloquées
}

```

```

private HashSet<AbilityType> _unlockedAbilities;

// Progression du joueur
private int _collectedEssence;
private Dictionary<string, bool> _completedSanctuaries;

// Implémentation...

public void UnlockAbility(AbilityType abilityType)
{
    if (_unlockedAbilities.Add(abilityType))
    {
        // Notifier et sauvegarder...
    }
}
}

```

3.4.2 Système de sauvegarde

Le système de sauvegarde utilise un format JSON pour stocker:

- Progression du joueur (capacités, collectibles)
- État du monde (zones purifiées, sanctuaires complétés)
- Position actuelle et checkpoint

```

public class SaveSystem : MonoBehaviour
{
    private const string SAVE_FILE_NAME = "gaia_save.json";

    public SaveData CurrentSave { get; private set; }

    // Implémentation...

    public void SaveGame()
    {

```

```

// Récupérer les données actuelles

SaveData saveData = new SaveData
{
    // Remplir avec les données actuelles...
};

// Sérialiser et sauvegarder
string json = JsonUtility.ToJson(saveData);
File.WriteAllText(GetSavePath(), json);
}

public bool LoadGame()
{
    string path = GetSavePath();
    if (File.Exists(path))
    {
        string json = File.ReadAllText(path);
        CurrentSave = JsonUtility.FromJson<SaveData>(json);
        return true;
    }
    return false;
}
}

```

4. Design Patterns et Bonnes Pratiques

4.1 Patterns implémentés

4.1.1 MVC (Modèle-Vue-Contrôleur)

Utilisation du MVC pour les entités principales:

```

// Modèle - Données

public class PlayerModel
{

```

```

// Attributs du joueur

public float Health { get; private set; }

public float Energy { get; private set; }

public Vector2 Velocity { get; set; }


// État actuel

public PlayerState CurrentState { get; set; }


// Événements

public event Action<float> OnHealthChanged;

public event Action<float> OnEnergyChanged;

public event Action<PlayerState> OnStateChanged;


// Implémentation...
}


// Vue - Représentation visuelle

public class PlayerView : MonoBehaviour

{
    // Composants graphiques

    [SerializeField] private Animator _animator;

    [SerializeField] private SpriteRenderer _renderer;

    [SerializeField] private ParticleSystem _moveParticles;


    // Méthodes d'animation et d'affichage

    public void UpdateAnimation(PlayerState state, Vector2 velocity)

    {
        // Mise à jour des paramètres d'animation...
    }


    // Implémentation...

```

```
}
```

```
// Contrôleur - Logique
```

```
public class PlayerController : MonoBehaviour
```

```
{
```

```
    [SerializeField] private PlayerModel _model;
```

```
    [SerializeField] private PlayerView _view;
```

```
    // Systèmes
```

```
    private MovementSystem _movement;
```

```
    private CombatSystem _combat;
```

```
    // Implémentation...
```

```
}
```

4.1.2 State Machine (Machine à états)

Utilisé pour la gestion des états du joueur et des ennemis:

```
public class StateMachine
```

```
{
```

```
    private IState _currentState;
```

```
    private Dictionary<Type, IState> _states = new Dictionary<Type, IState>();
```

```
    public void RegisterState(IState state)
```

```
{
```

```
        _states[state.GetType()] = state;
```

```
}
```

```
    public void ChangeState<T>() where T : IState
```

```
{
```

```
    if (_currentState != null)
```

```
    {
```

```
        _currentState.Exit();
```

```

    }

    _currentState = _states[typeof(T)];
    _currentState.Enter();
}

public void Update()
{
    _currentState?.Update();
}

public void FixedUpdate()
{
    _currentState?.FixedUpdate();
}
}

```

```

public interface IState
{
    void Enter();
    void Update();
    void FixedUpdate();
    void Exit();
}

```

4.1.3 Event-Driven Programming

Utilisation d'événements pour découpler les systèmes:

// Système d'événements centralisé

```

public static class EventManager
{
    // Dictionnaire d'événements génériques
    private static Dictionary<Type, Delegate> _events = new Dictionary<Type, Delegate>();
}

```

```

// Méthodes d'abonnement et de déclenchement

public static void Subscribe<T>(Action<T> listener) where T : struct
{
    // Implémentation...
}

public static void Unsubscribe<T>(Action<T> listener) where T : struct
{
    // Implémentation...
}

public static void Trigger<T>(T eventData) where T : struct
{
    // Implémentation...
}
}

// Exemple d'utilisation

public struct PlayerHealthChangedEvent
{
    public float CurrentHealth;
    public float PreviousHealth;
}

// Dans une classe abonnée

void OnEnable()
{
    EventManager.Subscribe<PlayerHealthChangedEvent>(OnPlayerHealthChanged);
}

```



```

void OnDisable()
{
    EventManager.Unsubscribe<PlayerHealthChangedEvent>(OnPlayerHealthChanged);
}

```

```

void OnPlayerHealthChanged(PlayerHealthChangedEvent evt)
{
    // Réagir au changement...
}

```

4.1.4 Singleton

Utilisé avec parcimonie pour les managers globaux:

```

public class GameManager : MonoBehaviour
{
    private static GameManager _instance;

    public static GameManager Instance
    {
        get
        {
            if (_instance == null)
            {
                Debug.LogError("GameManager not found in scene");
            }
            return _instance;
        }
    }

    private void Awake()
    {
        if (_instance != null && _instance != this)
        {

```

```

        Destroy(gameObject);

        return;
    }

```

```

        _instance = this;

        DontDestroyOnLoad(gameObject);
    }

```

```

// Implémentation...
}

```

4.1.5 Object Pooling

Utilisé pour optimiser l'instanciation d'objets fréquents:

```

public class ObjectPool<T> where T : Component
{
    private T _prefab;

    private List<T> _inactiveObjects = new List<T>();

    public ObjectPool(T prefab, int initialSize)
    {
        _prefab = prefab;

        for (int i = 0; i < initialSize; i++)
        {
            T obj = Object.Instantiate(_prefab);
            obj.gameObject.SetActive(false);
            _inactiveObjects.Add(obj);
        }
    }

    public T Get()
    {

```

```

    if (_inactiveObjects.Count > 0)
    {
        T obj = _inactiveObjects[0];
        _inactiveObjects.RemoveAt(0);
        obj.gameObject.SetActive(true);
        return obj;
    }

    return Object.Instantiate(_prefab);
}

public void Return(T obj)
{
    obj.gameObject.SetActive(false);
    _inactiveObjects.Add(obj);
}
}

4.2 Principes SOLID
4.2.1 Principe de responsabilité unique (S)
Chaque classe a une responsabilité unique et bien définie:

// Mauvais exemple: classe avec trop de responsabilités
public class BadPlayer : MonoBehaviour
{
    // Gère l'input, le mouvement, le combat, les animations, etc.
    // ...
}

// Bon exemple: responsabilités séparées
public class InputHandler : MonoBehaviour { /* ... */ }
public class MovementSystem : MonoBehaviour { /* ... */ }
public class CombatSystem : MonoBehaviour { /* ... */ }

```

```
public class AnimationController : MonoBehaviour { /* ... */ }
```

4.2.2 Principe d'ouverture/fermeture (O)

Les classes sont ouvertes à l'extension mais fermées à la modification:

// Classe de base pour les pouvoirs

```
public abstract class Ability
{
    public abstract string Name { get; }
    public abstract float EnergyCost { get; }
    public abstract float Cooldown { get; }

    public abstract void Activate(PlayerController player);
}
```

// Extensions sans modifier la classe de base

```
public class DashAbility : Ability
{
    public override string Name => "Dash";
    public override float EnergyCost => 10f;
    public override float Cooldown => 2f;

    public override void Activate(PlayerController player)
    {
        // Implémentation spécifique...
    }
}
```

```
public class RootWallAbility : Ability
{
    public override string Name => "Root Wall";
    public override float EnergyCost => 25f;
    public override float Cooldown => 5f;
```

```
public override void Activate(PlayerController player)
{
    // Implémentation spécifique...
}
}
```

4.2.3 Principe de substitution de Liskov (L)

Les classes dérivées peuvent remplacer leurs classes de base:

```
public abstract class Enemy
{
    public abstract void TakeDamage(float damage);
    public abstract void Attack(PlayerController target);
}
```

```
public class DroneEnemy : Enemy
{
    public override void TakeDamage(float damage)
    {
        // Implémentation spécifique...
    }
}
```

```
public override void Attack(PlayerController target)
{
    // Attaque à distance...
}
}
```

```
public class SentinelEnemy : Enemy
{
    public override void TakeDamage(float damage)
    {

```

```

        // Implémentation spécifique...
    }

    public override void Attack(PlayerController target)
    {
        // Attaque de mêlée...
    }
}

// Utilisation polymorphique
public void DamageAllEnemies(List<Enemy> enemies, float damage)
{
    foreach (Enemy enemy in enemies)
    {
        enemy.TakeDamage(damage); // Fonctionne pour tous les types d'ennemis
    }
}

```

4.2.4 Principe de ségrégation des interfaces (I)

Interfaces spécifiques plutôt qu'interfaces monolithiques:

// Mauvais exemple: interface trop large

```

public interface IBadGameObject
{
    void Update();
    void Render();
    void TakeDamage(float damage);
    void Attack(IDamageable target);
    void Move(Vector2 direction);
    void Jump();
    // ...
}

```

// Bon exemple: interfaces séparées par rôle

```
public interface IUpdatable
```

```
{
```

```
    void Update();
```

```
}
```

```
public interface IRenderable
```

```
{
```

```
    void Render();
```

```
}
```

```
public interface IDamageable
```

```
{
```

```
    void TakeDamage(float damage);
```

```
}
```

```
public interface IAttacker
```

```
{
```

```
    void Attack(IDamageable target);
```

```
}
```

```
public interface IMovable
```

```
{
```

```
    void Move(Vector2 direction);
```

```
}
```

// Une classe implémente uniquement les interfaces dont elle a besoin

```
public class Enemy : IUpdatable, IRenderable, IDamageable, IAttacker, IMovable
```

```
{
```

```
    // Implémentations...
```

```
}
```

```
public class Projectile : IUpdatable, IRenderable
{
    // Implémentations...
}
```

4.2.5 Principe d'inversion de dépendance (D)

Dépendances vis-à-vis des abstractions, pas des implémentations:

// Mauvais exemple: dépendance directe

```
public class BadPlayerController
{
    private AudioManager _audioManager; // Dépendance concrète

    public BadPlayerController()
    {
        _audioManager = new AudioManager(); // Couplage fort
    }

    public void Jump()
    {
        // Logique de saut...
        _audioManager.PlaySound("jump"); // Appel direct
    }
}
```

// Bon exemple: inversion de dépendance

```
public interface IAudioService
{
    void PlaySound(string soundId);
}
```

```
public class AudioManager : IAudioService
```



```

{
    public void PlaySound(string soundId)
    {
        // Implémentation...
    }
}

public class PlayerController
{
    private IAudioService _audioService; // Dépendance abstraite

    public PlayerController(IAudioService audioService)
    {
        _audioService = audioService; // Injection de dépendance
    }

    public void Jump()
    {
        // Logique de saut...
        _audioService.PlaySound("jump"); // Appel via interface
    }
}

```

4.3 Conventions de code

4.3.1 Conventions de nommage

- Classes et méthodes: PascalCase
- Variables et paramètres: camelCase
- Constantes: UPPER_SNAKE_CASE
- Champs privés: _camelCase (préfixés par underscore)
- Interfaces: IPascalCase (préfixées par 'I')
- Énumérations: PascalCase

```

public class PlayerController

```

```

{
    private const float MAX_HEALTH = 100f;

    [SerializeField] private float _moveSpeed;
    private Vector2 _currentVelocity;

    public float Health { get; private set; }

    public void ApplyDamage(float damageAmount)
    {
        // Implémentation...
    }
}

```

4.3.2 Organisation du code

- Un fichier par classe
- Regroupement des classes associées dans des namespaces
- Ordre des éléments dans une classe:
 1. Champs privés
 2. Propriétés publiques
 3. Constructeurs
 4. Méthodes Unity (Awake, Start, Update, etc.)
 5. Méthodes publiques
 6. Méthodes privées

```

namespace GaiaGuardian.Player
{
    public class PlayerController : MonoBehaviour
    {
        // Champs privés

        [SerializeField] private float _moveSpeed;
        private Vector2 _moveInput;
    }
}

```

```
// Propriétés publiques

public bool IsGrounded { get; private set; }

public PlayerState CurrentState { get; private set; }


// Constructeur (si applicable pour ScriptableObjects)


// Méthodes Unity

private void Awake()

{
    // Initialisation...
}


private void Start()

{
    // Configuration...
}


private void Update()

{
    // Logique mise à jour...
}


// Méthodes publiques

public void Jump()

{
    // Implémentation...
}


public void ApplyDamage(float amount)

{
    // Implémentation...
```

```

    }

    // Méthodes privées
    private void CheckGrounded()
    {
        // Implémentation...
    }
}
}

```

5. Gestion des Assets

5.1 Structure et organisation

5.1.1 Hiérarchie des dossiers

Assets/

```

├── Prefabs/           # Objets préfabriqués
|
| ├── Player/         # Préfabs liés au joueur
|
| ├── Enemies/        # Préfabs des ennemis
|
| ├── Environment/     # Éléments d'environnement
|
| ├── UI/             # Éléments d'interface
|
| └── FX/             # Effets visuels préfabriqués
|
├── Graphics/         # Assets graphiques
|
| ├── Characters/      # Sprites des personnages
|
| | ├── Gaia/         # Joueur principal
|
| | ├── Drones/       # Ennemis drones
|
| | ├── Sentinels/    # Sentinelles mécaniques
|
| | └── Bosses/       # Boss de fin de niveau
|
| ├── Environment/    # Décors et environnement
|
| | ├── Tilemaps/     # Sets de tuiles pour les niveaux
|
| | ├── Props/        # Objets décoratifs
|
| | └── Background/   # Arrière-plans parallaxe

```

		└─ Foreground/	# Premier plan
		└─ UI/	# Éléments d'interface utilisateur
		└─ FX/	# Effets visuels
		└─ Animations/	# Contrôleurs et clips d'animation
		└─ Audio/	# Assets audio
		└─ Music/	# Pistes musicales
		└─ SFX/	# Effets sonores
		└─ FMOD/	# Projet FMOD
		└─ Resources/	# Assets chargés dynamiquement

5.1.2 Conventions de nommage

Pour maintenir la cohérence et faciliter le travail d'équipe, nous suivons ces conventions de nommage:

- ****Préfixes descriptifs**** pour tous les fichiers:

- `PL\_` - Assets liés au joueur (Player)
- `EN\_` - Assets liés aux ennemis (Enemy)
- `ENV\_` - Assets d'environnement (Environment)
- `UI\_` - Éléments d'interface utilisateur
- `FX\_` - Effets visuels
- `SFX\_` - Effets sonores

- ****Suffixes fonctionnels**** indiquant le type d'asset:

- `\_SPR` - Sprite
- `\_ANM` - Animation
- `\_PFB` - Prefab
- `\_MAT` - Material
- `\_SHD` - Shader
- `\_SND` - Son

Exemple: `PL\_Dash\_ANM` (Animation du dash du joueur)

5.2 Pipeline des assets graphiques

5.2.1 Workflow Aseprite vers Unity

1. Création des sprites et animations dans Aseprite
2. Export en format PNG en utilisant le script d'exportation automatique
3. Import dans Unity avec les paramètres suivants:
 - Pixel Per Unit: 16
 - Filter Mode: Point (no filter)
 - Compression: None
4. Configuration des Sprite Sheets:
 - Mode: Multiple
 - Pivot: Bottom Center pour les personnages
 - Pivot: Center pour les objets

```
```csharp
```

```
// Exemple d'utilitaire d'importation automatisée des assets
```

```
public static class AsepriteImporter
```

```
{
```

```
 [MenuItem("Tools/Import Aseprite Files")]
```

```
 public static void ImportAsepriteFiles()
```

```
 {
```

```
 // Localiser tous les fichiers .ase dans le dossier d'import
```

```
 string[] asepritePaths = Directory.GetFiles(
```

```
 Application.dataPath + "/AsepriteSources",
```

```
 "*.ase",
```

```
 SearchOption.AllDirectories
```

```
);
```

```
 foreach (string asePath in asepritePaths)
```

```

{
 // Executer l'exportation via le CLI d'Aseprite
 ExportSpritesFromAseprite(asePath);

 // Configurer le sprite dans Unity
 ConfigureImportedSprite(asePath);
}
}

// Implémentation des méthodes d'assistance...
}

```

### 5.2.2 Gestion des atlas de sprites

Pour optimiser les performances de rendu, nous utilisons des atlas de sprites générés par Unity:

csharp

```

public class AtlasGenerator : MonoBehaviour
{
 [Serializable]
 public class AtlasDefinition
 {
 public string name;
 public List<Sprite> sprites;
 public int maxSize = 2048;
 }

 [SerializeField] private List<AtlasDefinition> _atlasDefinitions;

 [MenuItem("Tools/Generate Sprite Atlases")]
 public static void GenerateAtlases()
 {
 // Implémentation de génération d'atlas...
 }
}

```

```
}
```

### 5.3 Gestion audio avec FMOD

#### 5.3.1 Structure du projet FMOD

Le projet FMOD est organisé en banks:

- Master - Mixage global et paramètres audio principaux
- Music - Pistes musicales avec transitions entre états de purification
- SFX\_Player - Sons liés aux actions du joueur
- SFX\_Enemies - Sons liés aux ennemis
- SFX\_Environment - Sons d'ambiance et d'environnement
- UI - Sons de l'interface utilisateur

#### 5.3.2 Intégration Unity-FMOD

csharp

```
public class AudioManager : MonoBehaviour
```

```
{
```

```
 private static AudioManager _instance;
```

```
 public static AudioManager Instance => _instance;
```

```
 // Paramètres FMOD pour le contrôle dynamique de l'audio
```

```
 private FMOD.Studio.EventInstance _musicInstance;
```

```
 private FMOD.Studio.EventInstance _ambienceInstance;
```

```
 // Niveau de purification affectant la musique (0-100)
```

```
 private float _purificationLevel = 0f;
```

```
 private void Awake()
```

```
 {
```

```
 if (_instance != null && _instance != this)
```

```
 {
```

```
 Destroy(gameObject);
```

```
 return;
```

```
 }
```



```

 _instance = this;

 DontDestroyOnLoad(gameObject);

 InitializeAudio();
}

private void InitializeAudio()
{
 _musicInstance = FMODUnity.RuntimeManager.CreateInstance("event:/Music/LevelMusic");
 _ambianceInstance =
FMODUnity.RuntimeManager.CreateInstance("event:/SFX_Environment/Ambiance");

 _musicInstance.start();
 _ambianceInstance.start();
}

// Mise à jour des paramètres en fonction de l'état du jeu
public void UpdatePurificationLevel(float level)
{
 _purificationLevel = Mathf.Clamp(level, 0f, 100f);
 _musicInstance.setParameterByName("Purification", _purificationLevel);
 _ambianceInstance.setParameterByName("Purification", _purificationLevel);
}

// Jouer un son spécifique
public void PlaySound(string eventPath)
{
 FMODUnity.RuntimeManager.PlayOneShot(eventPath);
}

// Jouer un son à une position spatiale

```

```
public void PlaySound3D(string eventPath, Vector3 position)
{
 FMODUnity.RuntimeManager.PlayOneShot(eventPath, position);
}

// Nettoyage
private void OnDestroy()
{
 _musicInstance.stop(FMOD.Studio.STOP_MODE.ALLOWFADEOUT);
 _ambianceInstance.stop(FMOD.Studio.STOP_MODE.ALLOWFADEOUT);
 _musicInstance.release();
 _ambianceInstance.release();
}
}
```

---

## 6. Optimisation et Performance

### 6.1 Principes généraux d'optimisation

#### 6.1.1 Suivi des performances

Nous utilisons le Unity Profiler pour surveiller:

- Utilisation CPU
- Allocations mémoire
- Temps de rendu
- Charge de la physique

Les cibles de performance pour toutes les plateformes:

- 60 FPS constants
- Temps CPU par image < 16ms
- Drawcalls < 100 par image
- Utilisation mémoire < 500MB

#### 6.1.2 Optimisations de code critiques

csharp

*// Mauvais exemple: vérifications coûteuses dans Update*

```

private void BadUpdate()
{
 // Calcul coûteux à chaque frame
 foreach (GameObject enemy in GameObject.FindGameObjectsWithTag("Enemy"))
 {
 float distance = Vector3.Distance(transform.position, enemy.transform.position);
 if (distance < detectionRadius)
 {
 // Traitement...
 }
 }
}

```

*// Bon exemple: optimisé*

```

private List<Enemy> _enemiesInProximity = new List<Enemy>();
private float _updateTimer = 0f;
private const float UPDATE_INTERVAL = 0.25f; // Vérifier 4 fois par seconde seulement

```

```

private void GoodUpdate()
{
 _updateTimer += Time.deltaTime;

 if (_updateTimer >= UPDATE_INTERVAL)
 {
 _updateTimer = 0f;
 UpdateEnemiesInProximity();
 }
}

```

*// Utilisation de la liste pré-filtrée*

```

foreach (Enemy enemy in _enemiesInProximity)
{

```

```

 // Traitement plus efficace...
 }
}

private void UpdateEnemiesInProximity()
{
 _enemiesInProximity.Clear();

 Collider2D[] colliders = Physics2D.OverlapCircleAll(transform.position, detectionRadius,
 enemyLayerMask);

 foreach (Collider2D collider in colliders)
 {
 if (collider.TryGetComponent<Enemy>(out Enemy enemy))
 {
 _enemiesInProximity.Add(enemy);
 }
 }
}

```

## 6.2 Optimisation du rendu

### 6.2.1 Batching et atlas de sprites

Pour maximiser le batching:

- Toutes les tuiles de terrain utilisent le même matériau
- Les sprites sont regroupés en atlas par niveau/zone
- Utilisation de la hiérarchie statique pour les objets immobiles

csharp

```

public class SpriteOptimizer : MonoBehaviour
{
 [SerializeField] private Transform _containerTransform;

 [ContextMenu("Optimize Static Sprites")]
 public void OptimizeStaticSprites()
 {

```

```

 SpriteRenderer[] renderers =
 _containerTransform.GetComponentsInChildren<SpriteRenderer>();

 Dictionary<Material, List<SpriteRenderer>> materialGroups = new Dictionary<Material,
 List<SpriteRenderer>>();

 // Regrouper par matériau
 foreach (SpriteRenderer renderer in renderers)
 {
 if (!materialGroups.ContainsKey(renderer.material))
 {
 materialGroups[renderer.material] = new List<SpriteRenderer>();
 }

 materialGroups[renderer.material].Add(renderer);

 // Marquer comme statique pour le batching
 renderer.gameObject.isStatic = true;
 }

 Debug.Log($"Optimized {renderers.Length} sprites into {materialGroups.Count} batches");
 }
}

```

### 6.2.2 Occlusion et culling

csharp

```

public class DynamicObjectCulling : MonoBehaviour
{
 [SerializeField] private float _cullDistance = 20f;
 [SerializeField] private Transform _playerTransform;

 private List<CullableObject> _cullableObjects = new List<CullableObject>();
}

```

```
[System.Serializable]
```

```
private class CullableObject
```

```
{
```

```
 public Transform transform;
```

```
 public Behaviour[] componentsToDisable;
```

```
 public bool isActive = true;
```

```
}
```

```
private void Start()
```

```
{
```

```
 RegisterCullableObjects();
```

```
}
```

```
private void Update()
```

```
{
```

```
 UpdateCulling();
```

```
}
```

```
private void RegisterCullableObjects()
```

```
{
```

```
 // Trouver tous les objets avec le tag "Cullable"
```

```
 foreach (GameObject obj in GameObject.FindGameObjectsWithTag("Cullable"))
```

```
 {
```

```
 CullableObject cullable = new CullableObject
```

```
 {
```

```
 transform = obj.transform,
```

```
 componentsToDisable = obj.GetComponents<Behaviour>(),
```

```
 isActive = true
```

```
 };
```

```
 _cullableObjects.Add(cullable);
```

```

 }
}

private void UpdateCulling()
{
 foreach (CullableObject cullable in _cullableObjects)
 {
 float distance = Vector3.Distance(_playerTransform.position, cullable.transform.position);

 bool shouldBeActive = distance <= _cullDistance;

 if (shouldBeActive != cullable.isActive)
 {
 SetObjectActive(cullable, shouldBeActive);
 }
 }
}

```

```

private void SetObjectActive(CullableObject cullable, bool active)
{
 foreach (Behaviour component in cullable.componentsToDisable)
 {
 component.enabled = active;
 }

 cullable.isActive = active;
}
}

```

### 6.3 Optimisation de la physique

#### 6.3.1 Réglages des colliders et layers

Pour minimiser les calculs de physique:

- Utilisation de formes de collision simplifiées

- Configuration appropriée de la matrix de collision des layers
- Gestion des collisions par code pour les objets complexes

csharp

*// Configuration de la matrice de collision dans le code*

```
public class PhysicsLayerManager : MonoBehaviour
```

```
{
```

```
 private void Awake()
```

```
 {
```

```
 ConfigureCollisionMatrix();
```

```
 }
```

```
 private void ConfigureCollisionMatrix()
```

```
 {
```

```
 // Configuration des couches qui n'interagissent pas
```

```
 Physics2D.IgnoreLayerCollision(LayerMask.NameToLayer("Player"),
 LayerMask.NameToLayer("PlayerProjectile"), true);
```

```
 Physics2D.IgnoreLayerCollision(LayerMask.NameToLayer("Enemy"),
 LayerMask.NameToLayer("EnemyProjectile"), true);
```

```
 Physics2D.IgnoreLayerCollision(LayerMask.NameToLayer("PlayerProjectile"),
 LayerMask.NameToLayer("PlayerProjectile"), true);
```

```
 Physics2D.IgnoreLayerCollision(LayerMask.NameToLayer("EnemyProjectile"),
 LayerMask.NameToLayer("EnemyProjectile"), true);
```

```
 }
```

```
}
```

### 6.3.2 Raycasting optimisé

csharp

```
public class OptimizedGroundCheck : MonoBehaviour
```

```
{
```

```
 [SerializeField] private LayerMask _groundMask;
```

```
 [SerializeField] private Transform _feet;
```

```
 [SerializeField] private float _rayDistance = 0.2f;
```

```
// Cache des résultats
```



```

private RaycastHit2D[] _raycastHits = new RaycastHit2D[4];

private ContactFilter2D _contactFilter;

private void Awake()
{
 // Configurer le filtre de contact une seule fois
 _contactFilter = new ContactFilter2D
 {
 useLayerMask = true,
 layerMask = _groundMask,
 useTriggers = false
 };
}

public bool CheckIsGrounded()
{
 // Raycast unique avec résultats multiples
 int hitCount = Physics2D.Raycast(_feet.position, Vector2.down, _contactFilter, _raycastHits,
 _rayDistance);

 return hitCount > 0;
}
}

```

## 6.4 Gestion de la mémoire

### 6.4.1 Pooling d'objets

Nous utilisons un système de pooling extensif pour tous les objets créés/détruits fréquemment:

- Projectiles
- Effets de particules
- Ennemis
- Collectibles

csharp

*// Gestionnaire de pools centralisé*

```
public class PoolManager : MonoBehaviour
```

```

{
 private static PoolManager _instance;
 public static PoolManager Instance => _instance;

 [System.Serializable]
 public class Pool
 {
 public string tag;
 public GameObject prefab;
 public int size;
 }

 [SerializeField] private List<Pool> _pools;

 private Dictionary<string, Queue<GameObject>> _poolDictionary;

 private void Awake()
 {
 _instance = this;

 _poolDictionary = new Dictionary<string, Queue<GameObject>>();

 foreach (Pool pool in _pools)
 {
 Queue<GameObject> objectPool = new Queue<GameObject>();

 for (int i = 0; i < pool.size; i++)
 {
 GameObject obj = Instantiate(pool.prefab);
 obj.SetActive(false);
 objectPool.Enqueue(obj);
 }
 }
 }
}

```

```

 }

 _poolDictionary.Add(pool.tag, objectPool);
}
}

public GameObject SpawnFromPool(string tag, Vector3 position, Quaternion rotation)
{
 if (!_poolDictionary.ContainsKey(tag))
 {
 Debug.LogWarning($"Pool with tag {tag} doesn't exist.");
 return null;
 }

 GameObject objectToSpawn = _poolDictionary[tag].Count > 0
 ? _poolDictionary[tag].Dequeue()
 : Instantiate(_pools.Find(p => p.tag == tag).prefab);

 objectToSpawn.SetActive(true);
 objectToSpawn.transform.position = position;
 objectToSpawn.transform.rotation = rotation;

 IPoolable poolable = objectToSpawn.GetComponent<IPoolable>();
 if (poolable != null)
 {
 poolable.OnSpawnFromPool();
 }

 return objectToSpawn;
}

```

```

public void ReturnToPool(string tag, GameObject objectToReturn)
{
 if (!_poolDictionary.ContainsKey(tag))
 {
 Debug.LogWarning($"Pool with tag {tag} doesn't exist.");
 return;
 }

 objectToReturn.SetActive(false);

 IPoolable poolable = objectToReturn.GetComponent<IPoolable>();
 if (poolable != null)
 {
 poolable.OnReturnToPool();
 }

 _poolDictionary[tag].Enqueue(objectToReturn);
}
}

```

*// Interface pour les objets poolables*

```

public interface IPoolable
{
 void OnSpawnFromPool();
 void OnReturnToPool();
}

```

#### 6.4.2 Gestion des ressources

Stratégies pour réduire l'empreinte mémoire:

- Chargement asynchrone des scènes et ressources
- Déchargement des ressources inutilisées
- Compression des textures adaptée à chaque usage

csharp

```
public class ResourceManager : MonoBehaviour
```

```
{
```

```
 private Dictionary<string, UnityEngine.Object> _loadedResources = new Dictionary<string,
 UnityEngine.Object>();
```

```
 public T LoadResource<T>(string path) where T : UnityEngine.Object
```

```
{
```

```
 string key = $"{typeof(T).Name}_{path}";
```

```
 if (_loadedResources.ContainsKey(key))
```

```
{
```

```
 return _loadedResources[key] as T;
```

```
}
```

```
 T resource = Resources.Load<T>(path);
```

```
 if (resource != null)
```

```
{
```

```
 _loadedResources[key] = resource;
```

```
}
```

```
 return resource;
```

```
}
```

```
 public void UnloadUnusedResources()
```

```
{
```

```
 List<string> keysToRemove = new List<string>();
```

```
 foreach (var kvp in _loadedResources)
```

```
{
```

```
 // Vérifier si la ressource est encore utilisée
```

```
// Logique spécifique selon le type...
```

```
if (!IsResourceInUse(kvp.Value))
{
 keysToRemove.Add(kvp.Key);
}
}
```

```
foreach (string key in keysToRemove)
{
 _loadedResources.Remove(key);
}
```

```
// Forcer un nettoyage mémoire
```

```
Resources.UnloadUnusedAssets();
System.GC.Collect();
}
```

```
private bool IsResourceInUse(UnityEngine.Object resource)
```

```
{
```

```
// Logique de vérification selon le type
```

```
// Exemple simple pour un GameObject prefab
```

```
if (resource is GameObject)
```

```
{
```

```
 return GameObject.FindObjectsOfType<Transform>().Any(t =>
t.name.Contains(resource.name));
```

```
}
```

```
// Par défaut, considérer comme utilisé
```

```
return true;
```

```
}
```

```
}
```

---

## 7. Système de Compilation et Déploiement

### 7.1 Configuration de build

#### 7.1.1 Paramètres de compilation

csharp

```
public class BuildManager : EditorWindow
```

```
{
```

```
 private BuildPlayerOptions _buildPlayerOptions;
```

```
 private BuildTarget _selectedTarget;
```

```
 private string _buildPath;
```

```
 [MenuItem("Tools/Build Manager")]
```

```
 public static void ShowWindow()
```

```
 {
```

```
 GetWindow<BuildManager>("Build Manager");
```

```
 }
```

```
 private void OnGUI()
```

```
 {
```

```
 GUILayout.Label("Gaïa's Guardian Build Manager", EditorStyles.boldLabel);
```

```
 EditorGUILayout.Space();
```

```
 _selectedTarget = (BuildTarget)EditorGUILayout.EnumPopup("Build Target", _selectedTarget);
```

```
 _buildPath = EditorGUILayout.TextField("Build Path", _buildPath);
```

```
 EditorGUILayout.Space();
```

```
 if (GUILayout.Button("Build Development"))
```

```
 {
```

```

 BuildGame(BuildOptions.Development);
 }

 if (GUILayout.Button("Build Release"))
 {
 BuildGame(BuildOptions.None);
 }
}

private void BuildGame(BuildOptions options)
{
 _buildPlayerOptions = new BuildPlayerOptions
 {
 scenes = GetEnabledScenes(),
 locationPathName = _buildPath,
 target = _selectedTarget,
 options = options
 };

 BuildPipeline.BuildPlayer(_buildPlayerOptions);
}

private string[] GetEnabledScenes()
{
 List<string> sceneList = new List<string>();
 foreach (EditorBuildSettingsScene scene in EditorBuildSettings.scenes)
 {
 if (scene.enabled)
 {
 sceneList.Add(scene.path);
 }
 }
}

```



```

 }

 return sceneList.ToArray();
}
}

```

### 7.1.2 Configuration multi-plateforme

Nous utilisons des définitions conditionnelles pour gérer les différences entre plateformes:

csharp

```
public class PlatformSpecificManager : MonoBehaviour
```

```

{
 // Détection de plateforme

 public static bool IsMobile

 {
 get
 {
 #if UNITY_ANDROID || UNITY_IOS
 return true;
 #else
 return false;
 #endif
 }
 }

 // Initialisation spécifique à la plateforme

 private void Awake()
 {
 ApplyPlatformSpecificSettings();
 }

 private void ApplyPlatformSpecificSettings()
 {
 #if UNITY_STANDALONE

```

```

 // Configurations PC
 QualitySettings.vSyncCount = 1;
 Application.targetFrameRate = 60;
#elif UNITY_ANDROID || UNITY_IOS
 // Configurations mobile
 QualitySettings.vSyncCount = 0;
 Application.targetFrameRate = 30;
 // Réduire la qualité des effets
 ReduceEffectsQuality();
#endif
}

private void ReduceEffectsQuality()
{
 // Réduire la qualité des effets de particules pour les mobiles
 ParticleSystem[] systems = FindObjectsOfType<ParticleSystem>();
 foreach (ParticleSystem system in systems)
 {
 var main = system.main;
 main.maxParticles /= 2;
 }
}
}

```

## 7.2 Intégration continue (CI/CD)

### 7.2.1 Pipeline GitHub Actions

Nous utilisons GitHub Actions pour automatiser:

- Compilation des builds
- Exécution des tests unitaires
- Déploiement automatique des versions

yaml

*# .github/workflows/build.yml*

name: Build Gaïa's Guardian

on:

push:

branches: [ main, develop ]

pull\_request:

branches: [ main, develop ]

jobs:

buildForAllSupportedPlatforms:

name: Build for \${{ matrix.targetPlatform }}

runs-on: ubuntu-latest

strategy:

fail-fast: false

matrix:

targetPlatform:

- StandaloneWindows64
- StandaloneLinux64
- StandaloneOSX

steps:

- uses: actions/checkout@v2

with:

fetch-depth: 0

lfs: true

- uses: actions/cache@v2

with:

path: Library

key: Library-\${{ matrix.targetPlatform }}-\${{ hashFiles('Assets/\*\*', 'Packages/\*\*',  
'ProjectSettings/\*\*') }}

restore-keys: |

Library-`${{ matrix.targetPlatform }}`-

- uses: game-ci/unity-builder@v2

env:

UNITY\_LICENSE: `${{ secrets.UNITY_LICENSE }}`

with:

targetPlatform: `${{ matrix.targetPlatform }}`

- uses: actions/upload-artifact@v2

with:

name: Build-`${{ matrix.targetPlatform }}`

path: build/`${{ matrix.targetPlatform }}`

### 7.2.2 Tests automatisés

Les tests automatisés sont exécutés à chaque push:

yaml

*# .github/workflows/test.yml*

name: Test Gaïa's Guardian

on:

push:

branches: [ main, develop ]

pull\_request:

branches: [ main, develop ]

jobs:

testAllModes:

name: Test in `${{ matrix.testMode }}`

runs-on: ubuntu-latest

strategy:

fail-fast: false

matrix:

testMode:

- playmode
- editmode

steps:

- uses: actions/checkout@v2

with:

lfs: true

- uses: actions/cache@v2

with:

path: Library

key: Library-`{{ matrix.testMode }}`

restore-keys: |

Library-

- uses: game-ci/unity-test-runner@v2

env:

UNITY\_LICENSE: `{{ secrets.UNITY_LICENSE }}`

with:

testMode: `{{ matrix.testMode }}`

artifactsPath: `{{ matrix.testMode }}`-artifacts

- uses: actions/upload-artifact@v2

if: always()

with:

name: Test-`{{ matrix.testMode }}`

path: `{{ matrix.testMode }}`-artifacts

---

## 8. Tests et Debugging

### 8.1 Tests unitaires

#### 8.1.1 Structure des tests

Nous utilisons le cadre de test Unity Test Framework pour créer des tests unitaires:

```
csharp

using System.Collections;
using NUnit.Framework;
using UnityEngine;
using UnityEngine.TestTools;

namespace Tests
{
 public class PlayerControllerTests
 {
 private GameObject _playerObject;
 private PlayerController _controller;

 [SetUp]
 public void Setup()
 {
 // Créer un joueur de test
 _playerObject = new GameObject("TestPlayer");
 _controller = _playerObject.AddComponent<PlayerController>();

 // Configurer les dépendances pour les tests
 // ...
 }

 [TearDown]
 public void Teardown()
 {
 }
 }
}
```

```
// Nettoyer après chaque test
Object.Destroy(_playerObject);
}
```

```
[Test]
public void PlayerHealth_TakeDamage_ReducesHealth()
{
 // Arrange
 float initialHealth = 100f;
 float damageAmount = 25f;
 _controller.SetHealth(initialHealth);

 // Act
 _controller.TakeDamage(damageAmount);

 // Assert
 Assert.AreEqual(initialHealth - damageAmount, _controller.Health);
}
```

```
[UnityTest]
public IEnumerator PlayerMovement_Jump_IncreasesYPosition()
{
 // Arrange
 Vector3 initialPosition = _playerObject.transform.position;

 // Act
 _controller.Jump();

 // Wait for physics
 yield return new WaitForSeconds(0.1f);
```

```

 // Assert
 Assert.Greater(_playerObject.transform.position.y, initialPosition.y);
 }
}

```

### 8.1.2 Mocking et test doubles

Pour tester les composants avec des dépendances externes:

csharp

```

public interface IInputService
{
 bool IsJumpPressed();
 Vector2 GetMovementInput();
}

```

*// Implémentation réelle*

```

public class PlayerInputService : MonoBehaviour, IInputService
{
 public bool IsJumpPressed()
 {
 return Input.GetButtonDown("Jump");
 }

 public Vector2 GetMovementInput()
 {
 return new Vector2(Input.GetAxis("Horizontal"), 0);
 }
}

```

*// Mock pour les tests*

```

public class MockInputService : IInputService
{

```



```

public bool JumpPressed { get; set; }

public Vector2 MovementInput { get; set; }

public bool IsJumpPressed() => JumpPressed;
public Vector2 GetMovementInput() => MovementInput;
}

// Test utilisant le mock

[Test]
public void PlayerController_JumpInput_CallsJumpMethod()
{
 // Arrange
 var mockInput = new MockInputService { JumpPressed = true };
 _controller.SetInputService(mockInput);

 // Observer le comportement (via le spy)
 bool jumpMethodCalled = false;
 _controller.OnJump += () => jumpMethodCalled = true;

 // Act
 _controller.ProcessInput();

 // Assert
 Assert.IsTrue(jumpMethodCalled);
}

```

## 8.2 Outils de debugging

### 8.2.1 Système de logs avancé

csharp

```

public static class GaiaLogger
{
 public enum LogLevel

```

```
{
 Debug,
 Info,
 Warning,
 Error,
 Fatal
}
```

```
private static LogLevel _currentLogLevel = LogLevel.Debug;
```

```
public static void SetLogLevel(LogLevel level)
{
 _currentLogLevel = level;
}
```

```
public static void Log(LogLevel level, string message, Object context = null)
{
 if (level < _currentLogLevel)
 return;
```

```
 string formattedMessage = $"[{level}] {message}";
```

```
 switch (level)
 {
 case LogLevel.Debug:
 case LogLevel.Info:
 Debug.Log(formattedMessage, context);
 break;
 case LogLevel.Warning:
 Debug.LogWarning(formattedMessage, context);
 break;
```

```

 case LogLevel.Error:

 case LogLevel.Fatal:
 Debug.LogError(formattedMessage, context);
 break;
 }

 // Optionnel: journalisation vers un fichier
 LogToFile(level, message);
}

private static void LogToFile(LogLevel level, string message)
{
 #if !UNITY_EDITOR && !DEBUG
 // Logging to file only in builds
 // Implementation...
 #endif
}

// Méthodes pratiques

public static void Debug(string message, Object context = null) => Log(LogLevel.Debug,
message, context);

public static void Info(string message, Object context = null) => Log(LogLevel.Info, message,
context);

public static void Warning(string message, Object context = null) => Log(LogLevel.Warning,
message, context);

public static void Error(string message, Object context = null) => Log(LogLevel.Error, message,
context);

public static void Fatal(string message, Object context = null) => Log(LogLevel.Fatal, message,
context);
}

```

## 8.2.2 Outils de visualisation in-game

csharp

public class DebugVisualizer : MonoBehaviour

```

{
 [SerializeField] private bool _showDebugInfo = false;

 [Header("Display Options")]
 [SerializeField] private bool _showPlayerStats = true;
 [SerializeField] private bool _showFPS = true;
 [SerializeField] private bool _showMemoryUsage = true;
 [SerializeField] private bool _showColliders = false;

 private PlayerController _player;
 private float _deltaTime = 0.0f;
 private GUIStyle _guiStyle = new GUIStyle();

 private void Start()
 {
 _player = FindObjectOfType<PlayerController>();

 // Style de texte
 _guiStyle.fontSize = 20;
 _guiStyle.normal.textColor = Color.white;
 _guiStyle.fontStyle = FontStyle.Bold;
 }

 private void Update()
 {
 _deltaTime += (Time.unscaledDeltaTime - _deltaTime) * 0.1f;

 // Activer/désactiver avec une touche
 if (Input.GetKeyDown(KeyCode.F3))
 {
 _showDebugInfo = !_showDebugInfo;
 }
 }
}

```

```

 }
}

private void OnGUI()
{
 if (!_showDebugInfo)
 return;

 int y = 10;

 if (_showFPS)
 {
 float fps = 1.0f / _deltaTime;

 GUI.Label(new Rect(10, y, 200, 20), $"FPS: {Mathf.Round(fps)}", _guiStyle);

 y += 25;
 }

 if (_showMemoryUsage)
 {
 float memoryUsage = System.GC.GetTotalMemory(false) / (1024f * 1024f);

 GUI.Label(new Rect(10, y, 300, 20), $"Memory: {memoryUsage:F2} MB", _guiStyle);

 y += 25;
 }

 if (_showPlayerStats && _player != null)
 {
 GUI.Label(new Rect(10, y, 300, 20), $"Health: {_player.Health:F0}/{_player.MaxHealth:F0}",
_guiStyle);

 y += 25;

 GUI.Label(new Rect(10, y, 300, 20), $"Energy: {_player.Energy:F0}/{_player.MaxEnergy:F0}",
_guiStyle);

 y += 25;
 }
}

```

```

 GUI.Label(new Rect(10, y, 300, 20), $"Position: {_player.transform.position}", _guiStyle);
 y += 25;
 }
}

private void OnDrawGizmos()
{
 if (!_showDebugInfo || !_showColliders)
 return;

 // Visualiser les colliders

 Collider2D[] colliders = FindObjectsOfType<Collider2D>();
 foreach (Collider2D collider in colliders)
 {
 Gizmos.color = Color.green;

 if (collider is BoxCollider2D)
 {
 BoxCollider2D boxCollider = collider as BoxCollider2D;
 Gizmos.matrix = boxCollider.transform.localToWorldMatrix;
 Gizmos.DrawWireCube(boxCollider.offset, boxCollider.size);
 }
 else if (collider is CircleCollider2D)
 {
 CircleCollider2D circleCollider = collider as CircleCollider2D;
 Gizmos.matrix = circleCollider.transform.localToWorldMatrix;
 Gizmos.DrawWireSphere(circleCollider.offset, circleCollider.radius);
 }
 }
}
}

```

---

## 9. API de Référence

### 9.1 Système de jeu principal

#### 9.1.1 GameManager

Le GameManager est le point central de contrôle du jeu, gérant l'état global et la coordination des systèmes.

#### Méthodes:

csharp

*// Initialise le jeu avec les paramètres spécifiés*

```
public void Initialize(GameSettings settings)
```

*// Démarre une nouvelle partie*

```
public void StartNewGame()
```

*// Charge une partie sauvegardée*

```
public bool LoadGame(int saveSlot)
```

*// Sauvegarde la partie en cours*

```
public void SaveGame(int saveSlot)
```

*// Met le jeu en pause*

```
public void PauseGame(bool isPaused)
```

*// Change le niveau actuel*

```
public void ChangeLevel(string levelName, Vector2 spawnPosition)
```

*// Gère la mort du joueur*

```
public void HandlePlayerDeath()
```

*// Réinitialise le joueur au dernier checkpoint*

```
public void RespawnPlayerAtCheckpoint()
```

**Propriétés:**

csharp

*// État actuel du jeu*

```
public GameState CurrentGameState { get; private set; }
```

*// Niveau de jeu actuel*

```
public LevelManager CurrentLevel { get; private set; }
```

*// Référence au controller du joueur*

```
public PlayerController Player { get; private set; }
```

*// Indique si le jeu est en pause*

```
public bool IsPaused { get; private set; }
```

*// Niveau global de purification (0-100)*

```
public float GlobalPurificationLevel { get; private set; }
```

### 9.1.2 PlayerController

Le PlayerController gère toutes les actions et l'état du personnage principal.

**Méthodes:**

csharp

*// Applique des dégâts au joueur*

```
public void TakeDamage(float amount, Vector2 knockbackDirection)
```

*// Restaure de la santé*

```
public void Heal(float amount)
```

*// Fait sauter le personnage*

```
public void Jump()
```

*// Effectue un dash dans la direction spécifiée*

```
public void Dash(Vector2 direction)
```



*// Active une attaque de mêlée*

public void MeleeAttack()

*// Active une attaque à distance*

public void RangedAttack()

*// Active une capacité spéciale*

public void ActivateAbility(AbilityType abilityType)

*// Interagit avec l'objet devant le joueur*

public void Interact()

*// Purification d'une zone*

public void PurifyArea(PurificationNode node)

### **Propriétés:**

csharp

*// Santé actuelle et maximale*

public float Health { get; private set; }

public float MaxHealth { get; private set; }

*// Énergie actuelle et maximale*

public float Energy { get; private set; }

public float MaxEnergy { get; private set; }

*// État actuel du joueur*

public PlayerState CurrentState { get; private set; }

*// Direction actuelle du mouvement*

public Vector2 MoveDirection { get; private set; }

*// Vitesse actuelle du mouvement*

```
public float CurrentSpeed { get; private set; }
```

*// Indique si le joueur est au sol*

```
public bool IsGrounded { get; private set; }
```

*// Capacités débloquées*

```
public HashSet<AbilityType> UnlockedAbilities { get; private set; }
```

## 9.2 Système d'ennemis

### 9.2.1 EnemyController

Classe de base pour tous les types d'ennemis.

#### **Méthodes:**

csharp

*// Initialise l'ennemi avec les paramètres spécifiés*

```
public virtual void Initialize(EnemySettings settings)
```

*// Applique des dégâts à l'ennemi*

```
public virtual void TakeDamage(float amount, Vector2 hitDirection)
```

*// Active le comportement d'attaque*

```
public virtual void Attack()
```

*// Étourdit temporairement l'ennemi*

```
public virtual void Stun(float duration)
```

*// Applique un effet d'affaiblissement dû à la purification*

```
public virtual void ApplyPurificationEffect(float purificationLevel)
```

*// Déplace l'ennemi vers la position cible*

```
public virtual void MoveTowards(Vector2 targetPosition, float speed)
```

#### **Propriétés:**

```
csharp
```

```
// Santé actuelle et maximale
```

```
public float Health { get; protected set; }
```

```
public float MaxHealth { get; protected set; }
```

```
// État actuel de l'ennemi
```

```
public EnemyState CurrentState { get; protected set; }
```

```
// Type d'ennemi
```

```
public EnemyType Type { get; protected set; }
```

```
// Statistiques de combat
```

```
public float AttackDamage { get; protected set; }
```

```
public float AttackRange { get; protected set; }
```

```
public float AttackSpeed { get; protected set; }
```

```
// Perception du joueur
```

```
public bool CanSeePlayer { get; protected set; }
```

```
public Vector2 LastKnownPlayerPosition { get; protected set; }
```

### 9.3 Système d'environnement

#### 9.3.1 LevelManager

Gère l'état et les fonctionnalités d'un niveau.

#### **Méthodes:**

```
csharp
```

```
// Initialise le niveau
```

```
public void Initialize()
```

```
// Charge tous les éléments du niveau
```

```
public void LoadLevel()
```

```
// Décharge le niveau et libère les ressources
```

```
public void UnloadLevel()
```

```
// Active un checkpoint
```

```
public void ActivateCheckpoint(string checkpointId)
```

```
// Purifie une zone du niveau
```

```
public void PurifyZone(string zoneId, float purificationAmount)
```

```
// Fait apparaître un ennemi à la position spécifiée
```

```
public GameObject SpawnEnemy(EnemyType type, Vector2 position)
```

```
// Ouvre un passage vers un autre niveau
```

```
public void OpenLevelTransition(string targetLevelId, string entryPointId)
```

```
// Débloque une capacité pour le joueur
```

```
public void UnlockAbility(AbilityType abilityType)
```

### **Propriétés:**

```
csharp
```

```
// Identifiant unique du niveau
```

```
public string LevelId { get; private set; }
```

```
// État actuel du niveau
```

```
public LevelState CurrentState { get; private set; }
```

```
// Checkpoint actif
```

```
public Checkpoint ActiveCheckpoint { get; private set; }
```

```
// Niveau de purification global du niveau (0-100)
```

```
public float PurificationLevel { get; private set; }
```

```
// Zones de purification du niveau
```

```
public Dictionary<string, PurificationZone> PurificationZones { get; private set; }
```

```
// Collectibles trouvés
```

```
public List<string> FoundCollectibles { get; private set; }
```

### 9.3.2 PurificationSystem

Gère la purification de l'environnement et ses effets.

#### **Méthodes:**

```
csharp
```

```
// Initialise le système avec les zones du niveau
```

```
public void Initialize(List<PurificationZone> zones)
```

```
// Purifie une zone spécifique
```

```
public void PurifyZone(string zoneId, float amount)
```

```
// Met à jour l'état visuel de l'environnement basé sur le niveau de purification
```

```
public void UpdateEnvironmentVisuals()
```

```
// Applique les effets de purification aux ennemis dans une zone
```

```
public void ApplyPurificationEffects(string zoneId)
```

```
// Vérifie si toutes les zones sont purifiées
```

```
public bool IsLevelFullyPurified()
```

```
// Réinitialise le système pour un nouveau niveau
```

```
public void Reset()
```

#### **Propriétés:**

```
csharp
```

```
// Niveau de purification global (0-100)
```

```
public float GlobalPurificationLevel { get; private set; }
```

```
// État de purification par zone
```

```
public Dictionary<string, float> ZonePurificationLevels { get; private set; }
```

```
// Événements de purification
```

```
public event Action<float> OnGlobalPurificationChanged;
```

```
public event Action<string, float> OnZonePurificationChanged;
```

---

## 10. Annexes

### 10.1 Diagrammes UML

Les diagrammes UML complets sont disponibles dans les fichiers suivants:

- Diagrams/ClassDiagram.png - Structure générale des classes
- Diagrams/StateChart.png - Diagramme d'état pour le joueur et les ennemis
- Diagrams/SequenceDiagram.png - Flux d'interaction entre les systèmes principaux

### 10.2 Cheatsheet des contrôles pour les développeurs

# Contrôles du joueur (configuration par défaut)

- ZQSD / Flèches directionnelles - Mouvement
- Espace - Saut (double saut si pressé deux fois)
- Shift - Dash aérien
- E - Interaction
- Clic gauche - Attaque de mêlée
- Clic droit - Attaque à distance (Vortex de pollen)
- 1-4 - Sélection des pouvoirs
- F - Utilisation du pouvoir sélectionné
- C - Camouflage
- Ctrl - Glissade

# Commandes de debug (mode développeur uniquement)

- F1 - Active/désactive le mode God (invincibilité)
- F2 - Active/désactive les collisions
- F3 - Affiche/masque les informations de debug
- F4 - Recharge le niveau actuel
- F5 - Sauvegarde rapide

- F6 - Charge la dernière sauvegarde
- F7 - Tue tous les ennemis du niveau
- F8 - Débloque toutes les capacités
- F9 - Téléporte au checkpoint suivant
- F10 - Purification instantanée de la zone actuelle

### 10.3 Résolution des problèmes courants

Problème	Cause possible	Solution
Freezes sur les transitions de niveau	Chargement synchrone des ressources	Convertir en chargement asynchrone avec LoadSceneAsync
Fuites mémoire avec les particules	Particules non détruites	Utiliser le système de pooling et vérifier les références
Ralentissements avec nombreux ennemis	Trop de calculs physiques simultanés	Implémenter un système de LOD pour la physique
Animations qui se bloquent	Transitions incorrectes dans l'Animator	Vérifier les conditions de transition et les paramètres
Collisions fantômes	Problème de hitbox ou de layers	Vérifier la matrice de collision et la taille des colliders

### 10.4 Glossaire des termes techniques

- **FSM (Finite State Machine):** Système de gestion d'états pour le comportement des entités
- **LOD (Level of Detail):** Technique d'optimisation réduisant la complexité des objets distants
- **Pooling:** Réutilisation d'objets pour éviter les opérations d'instanciation/destruction coûteuses
- **Raycasting:** Technique de détection utilisant des "rayons" pour identifier les objets
- **Hitbox:** Zone de collision utilisée pour la détection des coups
- **Sprite Atlas:** Regroupement de textures pour optimiser le rendu
- **Coroutine:** Fonction pouvant être suspendue et reprise, utilisée pour des opérations asynchrones
- **Tilemap:** Système de création de niveaux basé sur des tuiles
- **Shader:** Programme exécuté sur le GPU pour le rendu graphique
- **Prefab:** Modèle d'objet réutilisable dans Unity